



School of Informatics of AUTH

Postgraduate Studies Program

Artificial Intelligence

Poverty Prediction Challenge Machine Learning Project

Georgios Nikolis

AEM: 209

gnikolia@csd.auth.gr

January 2026

Contents

1	Introduction	2
2	Data Pre-processing	2
3	Data Analysis	2
4	Machine Learning Algorithms	4
5	Validation	5
6	Results	5
7	Conclusion	6

1 Introduction

This project was part of my submission for the Poverty Prediction Challenge by driven-data.org. The primary objective is to predict per capita household consumption and the corresponding poverty rates using historical survey data. This report presents the complete machine learning pipeline, including data pre-processing techniques and the comparative evaluation of multiple models. The implementation was developed in Python, using libraries such as scikit-learn and tensorflow, with the full code available in the repository. This workflow demonstrates the recommended methodology for addressing real-world machine learning challenges.

2 Data Pre-processing

The features file, as well as their description were analyzed to get a basic understanding. This analysis revealed that several variables were irrelevant for consumption prediction. Moreover, we examined the creation of some other features by combining the ones that we already have, aiming to enhance model performance.

To begin with, features 'utl_exp_ppp17', 'weight', 'hhid', 'survey_id', 'strata' and 'com' were removed as they were found unable to contribute. Moreover, the features 'num_children5', 'num_children10', 'num_elderly' were summed and divided with 'hsize' in order to create a feature that quantifies the economic burden of family members who are most likely not earning a salary. This feature was called 'dependency_ratio'.

In addition, the 'worker_share' feature was engineered by dividing 'sworkershh' with 'hsize'. This feature quantifies the percentage of the family members that are actually working. Furthermore, a feature called 'housing_index' was calculated which measures basic living standards by summing utility features like 'water', 'toilet' etc. Another engineered feature is the log transformation of the variable 'hsize', since costs don't increase linearly. For instance, 5 people do not spend 5 times more than 1 person, due to sharing.

The following step was a crucial one, as we had to properly convert categorical features into numerical. There are some standard ways to do this. However, a hybrid approach was chosen, as the dataset contains some features whose rank matters, like education, while it also contains other features that need to just be encoded into numbers. For this purpose, we ranked some of the features and assigned a number to them. To begin with, education was ranked from 0 to 6, which corresponds to the level of education a person has. For instance, 'Never attended' was 0, while 'Complete Tertiary Education' was 6. This process not only converts the values of the features into numerical values that models can use but also take into account the significance of each response. A similar process was followed for the variables regarding 'urban', 'water source', 'sanitation', 'employment', 'nonargic' and binary. The 'housing_index' was assigned a score. Finally, the rest of the categorical features were one-hot-encoded.

The final training dataset consists of 72 features and 104232 observations. The exact steps were performed for the test dataset as well. The features contained a diverse range of information regarding the size of the household, the income of its members and their quality of life.

3 Data Analysis

Figure 1 shows the distribution of household consumption. We can observe that it follows a positively skewed distribution. This graph can give us an intuitive sense of the range that

consumption values follow and what to expect in terms of predictions. In addition, it could provide insight into the pre-processing techniques that could be followed.

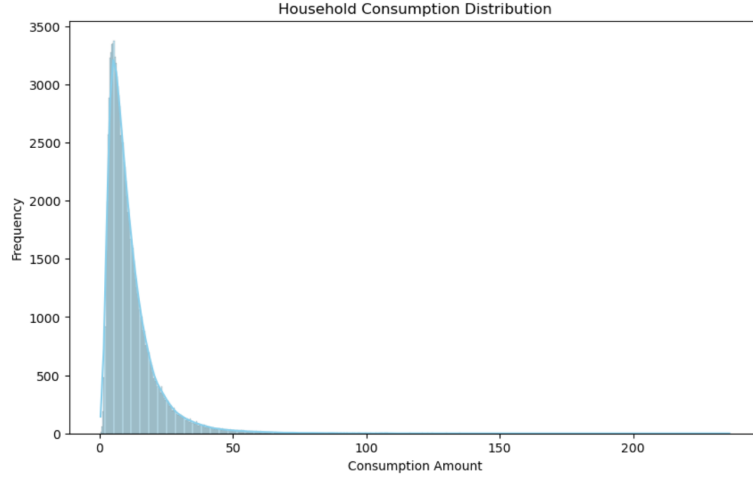


Figure 1: Consumption Distribution.

The following correlation heatmap (Figure 2) can be used to quickly identify the features that matter the most. The red colors represent strong positive correlations, while blue colors represent strong negative correlations. At a glance, we can see that there are no strong correlations between consumption and the other variables, which is an indication that predicting consumption might require non-linear models. The strongest feature is the education index, while the housing index is also close. We can also observe the dependency ratio has strong correlation with the number of elderly, which suggests that feature engineering was successfully.

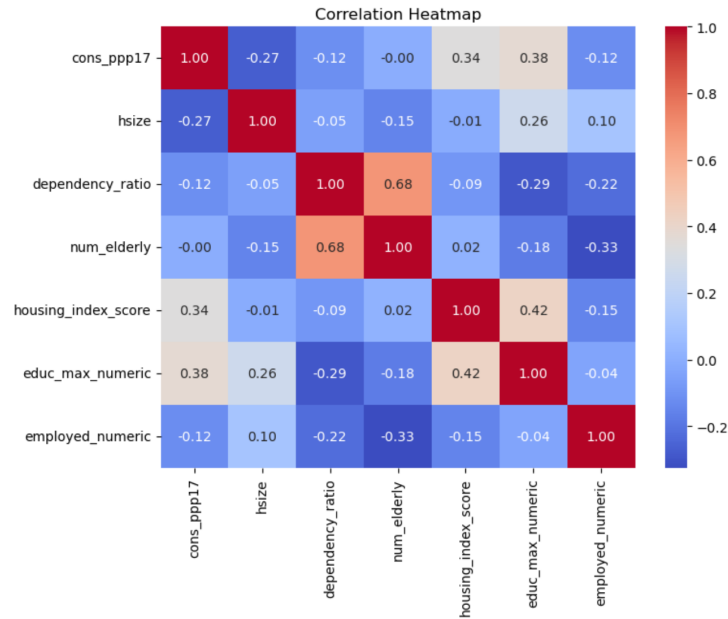


Figure 2: Correlation Heatmap of some features.

Finally, Figure 3 shows the average per capita consumption by family size. As expected, the per capita consumption drops as the number of people in the household increases. This validates the accuracy of the dataset, as people share resources in larger families. Consequently, the log transformation of the variable ‘hsize’ makes sense.

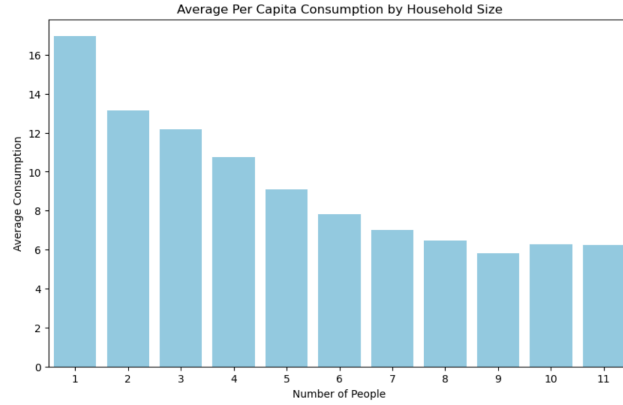


Figure 3: Per capita consumption by houshold size.

4 Machine Learning Algorithms

The core goal of this application is to predict the expected consumption. The input dataset contains a variety of socio-economic variables, while some of them are one-hot-encoded. These are important considerations in the selection of the model or algorithm that will be used. To begin with, the Linear Regression model was used as a baseline. In general, it is a good practice to begin with simple approaches and increase their complexity to find the best possible solution. Linear Regression is a straightforward and easy to interpret method. However, it has certain drawbacks, such as the fact that it assumes the target variable is a linear process. Nevertheless, this method was used as a starting point.

Another method that was used is the Random Forest Regressor. Spending habits is a complex issue and the ensemble architecture of the RFR could be an important tool. Moreover, it could help in detecting and dealing with non-linearity, which is a limitation of the previous approach. A grid search to find the best parameters was performed. Specifically, number of estimators possible values was 100 or 200, while the max depth of the trees could be among 10, 20 or None. Finally, the ‘min_samples_leaf’ parameter could take values 1 or 4. The results after 5 iterations 3 validation procedures showcased that 100 estimators, 4 samples for splitting and None as depth were the optimal selections.

Furthermore, the XGBoost Regressor was utilized. It uses a sequential approach when building trees, in contrast to the RFR. In essence, every new tree tries to fix the errors of the previous one. This could be seen as a more evolved version of the previous approach. The grid search was conducted over the parameter space defined by: $n_estimators \in \{100, 300\}$, $learning_rate \in \{0.01, 0.1\}$, $max_depth \in \{3, 6, 10\}$, and $subsample \in \{0.7, 1.0\}$. The optimal model configuration was identified as: $n_estimators = 100$, $max_depth = 6$, $learning_rate = 0.1$, and $subsample = 0.7$.

For the XGBoost model, another optimization method was also examined. The optuna package is an automated hyperparameter library. Its main advantage is that it learns from the ‘mistakes’ of previous iterations and does not attempt to brute-force all the possible combinations of parameters, reducing time of execution significantly. A diverse set of possible values for many hyperparameters was examined in the experiment. The final choice is presented in Table 1a. We can see that the numbers are close to those of the basic grid search. However, these mild differences, as well as the addition of the extra parameters, may be crucial.

The deep learning model architecture that was utilized had 3 hidden layers. Dropout and early stopping were considered for training. After testing a baseline architecture, the keras

Table 1: Model Configurations

(a) XGBoost Hyperparameters (Optuna)

Hyperparameter	Value
Learning Rate	0.0452
Max Depth	7
Subsample	0.8999
L1 Regularization	4.2413
L2 Regularization	7.6287

(b) DL Architecture

Layer (Units/Rate)
Dense (128)
Dropout (0.2)
Dense (64)
Dense (32)
Dense (1)

tuner was used in order to find the best set of hyperparameters. A simplified version of the selected architecture is presented in Table 1b.

5 Validation

The initial dataset was split into training and testing sets. The hyperparameter turning methods split the training into further sub-training and sub-validation sets, so as to avoid leakage of information. This was done due to the fact that the test data that was provided by the competition was unlabeled and therefore could not be used in the training or the validation procedure.

6 Results

In Table 2 we can see some core metrics from all the previously described models. We can see that the best model in terms of MSE and MAE is the XGBoost Regressor, while it also has the highest R2. This means that this architecture captures information in the most efficient way. This model was tuned with a grid search. The optuna tuned model slightly underperforms. This does not necessarily mean that the optimization was not successful, but that the experiment was overfitted on the validation set that was used for this test.

Table 2: Comparison of Models

Model	MSE	MAE	R^2
LinearRegression	49.54	4.20	0.52
RandomForestRegressor	42.91	3.69	0.58
XGBRegressor	40.77	3.58	0.61
XGBRegressor_optuna	44.56	3.76	0.57
Base Deep Learning Model	42.17	3.82	0.59
Tuned Deep Learning Model	42.02	3.61	0.59

The deep learning models showcase good performance, with a higher R2 than the Random Forest. In addition, tuning the model improved the MAE and MSE, making it the second most accurate after the XGBoost. Finally, the Linear Regression had the lowest score as expected, due to the visualizations that were presented earlier, which suggested that a linear model might be insufficient.

Figure 4 shows a screenshot of 3 submissions on the competition website. We can see that the Deep Learning model outperforms the XGBoost grid search model, as well as the optuna tuned one. This suggests that while the XGBoost models performed well during training, the Deep Learning architecture was more effective at capturing the patterns required by the competition’s final evaluation criteria. The best ranking achieved was 161 out of a total of 315, as seen in Figure 5.

Public score	Who	Details
15.858	 giorgosnn	id-300313 · 22min ago · xgboost with grid search
9.717	 giorgosnn	id-300316 · 10min ago · deep learning tuned model
23.388	 giorgosnn	id-300317 · 1min ago · xgb optuna

Figure 4: Submission Scores.

Best score 9.717		Current rank #161		Submissions used 3 of 3	
#159	  2w 2d ago · 7 submissions	9.524		6.878	
#160	  2w 2d ago · 2 submissions	9.716		6.888	
#161	 giorgosnn 9min ago · 3 submissions	9.717		6.499 Share your work?	
#162	  20h ago · 7 submissions	9.928		6.533	
#163	  5d 13h ago · 4 submissions	9.943		5.802	

Figure 5: Submission Ranking.

7 Conclusion

This report presented the design of the solution for the Poverty Prediction Challenge by drivendata.org. Initially, the data was pre-processed, visualized and new features were engineered. Afterwards, a baseline model was trained. Every new model was addressing a weakness of the previous one. Grid searches were implemented to find the best possible parameters for each solution, while the XGBoost and the Deep Learning models were also tuned with the use of specialized optimizers. The results show that the Deep Learning model scored the highest at the competition leaderboard, despite the fact the XGBoost model showcased the highest performance on the internal test set. This comprehensive approach outlines the best practices for solving machine learning problems in a real-world context.